

环境配置和系统编译烧录文档

本文档说明 ws63_ohos 源码的环境配置、编译流程、烧录步骤以及 hb 构建工具的完整使用方法。

文档更新于2026-03-12，为本人在开发中总结的经验，如有问题请及时联系王老师 (wangpeizheng@sztu.edu.cn)。

一、开发环境配置

1.1 方式一：原生 Linux 环境（不推荐自己配置环境）

在 Ubuntu 或其它 Linux 发行版下直接开发，需满足：

- Ubuntu 20.04 或更高版本（推荐 64 位）
- Python 3.7.4 及以上
- 磁盘空间充足（建议 100GB 以上）

1.2 方式二：WSL 子系统环境（Windows 用户，推荐）

适用于 Windows 用户，通过 WSL 2 运行 Linux 子系统进行开发。

系统要求

系统要求	说明
Windows 10 版本 2004 及以上（内部版本 19041 及以上）或 Windows 11	可通过 Win+R 输入 <code>winver</code> 查看版本
x64 系统 Linux 内核更新包	wsl_update_x64.msi

手工安装步骤

1. 启用 WSL 功能

以**管理员身份**打开 PowerShell，执行：

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

2. 启用虚拟机平台

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

3. 安装 Linux 内核更新包

- 下载并安装 [wsl_update_x64.msi](#)
- **重启电脑**（必须重启，否则后续可能报错）

4. 下载并导入 fbb_ws63_wsl 发行包

- 下载链接：https://hispark-obs.obs.cn-east-3.myhuaweicloud.com/fbb_ws63_wsl.tar
- 在非系统盘新建目录（如 `D:/ubuntu/`）
- 以**管理员身份**打开 PowerShell，执行（路径按实际修改）：

```
wsl --import fbb_ws63 D:/ubuntu/ C:/Users/Administrator/Desktop/fbb_ws63_wsl.tar
```

- 导入完成后**再次重启电脑**（避免卡顿）
- 验证安装：`wsl --list`，应能看到 `fbb_ws63`

5. 安装 VSCode 及远程开发扩展

- 安装 [VSCode](#)
- 在扩展市场搜索并安装 **Remote Development**
- 可选：安装 **Chinese (Simplified)** 实现中文界面
- 推荐下载 [TRAE](#)，利用AI Coding工具进行嵌入式编程，效率更高。

6. 连接 WSL 开发

- 打开 VSCode → 远程资源管理器 → 选择 `fbb_ws63` → 在当前窗口中连接
- 连接成功后，新建终端即可在 WSL 中执行编译、烧录等操作

推荐使用TRAE等AI Coding工具来查看和编写本实验涉及的代码。

二、一键下载和配置开发环境（推荐）

本节提供一键脚本，适用于快速搭建开发环境。

2.1 脚本位置与用法

项目	说明
脚本路径	<code>/root/download_ws63_ohos.sh</code> （将脚本复制到 <code>/root</code> 目录下）
用法	<code>cd /root && ./download_ws63_ohos.sh [安装目录]</code>
安装目录	可选，不指定时默认安装到 <code>\$HOME/ws63_ohos</code> ，不建议指定安装目录，直接使用默认

示例：

```
# 安装到默认目录 $HOME/ws63_ohos
cd /root
chmod +x ./download_ws63_ohos.sh
./download_ws63_ohos.sh
```

2.2 脚本执行内容

脚本依次完成以下步骤：

步骤	操作
1	安装 repo 工具 (含 pip3 requests)
2	安装 git-lfs
3	配置 repo 环境变量 (~/.bin 加入 PATH)
4	通过 repo 下载源码 (init、sync、lfs pull)
5	下载预编译工具链 (./build/prebuilts_download.sh)
6	安装 hb 构建工具
7	安装 cmake
8	将 RISC-V 工具链路径写入 ~/.bashrc

2.3 脚本执行完成后

脚本结束时提示：

1. 使环境变量生效： `source ~/.bashrc`
2. 验证工具链： `riscv32-linux-musl-gcc -v`
3. 编译系统：进入源码目录执行 `hb set -p nearlink_dk_3863` 和 `hb build -f`

2.4 脚本源码

```
#!/bin/bash
#=====
# ws63_ohos 一键下载和配置开发环境脚本
# 用法: bash /root/download_ws63_ohos.sh [安装目录]
#       不指定目录时，默认安装到 $HOME/ws63_ohos
#=====

set -e

TARGET_DIR="${1:-$HOME/ws63_ohos}"
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"

echo "=====
echo "  ws63_ohos 一键下载和配置开发环境"
echo "=====
echo "安装目录: $TARGET_DIR"
echo ""

#-----
# 1. 安装 repo 工具
#-----

echo "[1/7] 安装 repo 工具..."
mkdir -p ~/bin
if [ ! -f ~/bin/repo ]; then
    curl -sL https://gitee.com/oschina/repo/raw/fork_flow/repo-py3 -o ~/bin/repo
    chmod a+x ~/bin/repo
fi
```

```
pip3 install -q -i https://repo.huaweicloud.com/repository/pypi/simple requests
2>/dev/null || true
echo "  ✓ repo 已就绪"

#-----
# 2. 安装 git-lfs
#-----
echo "[2/7] 安装 git-lfs..."
if ! command -v git-lfs &>/dev/null; then
    sudo apt-get update -qq
    sudo apt-get install -y -qq git-lfs
fi
git lfs install 2>/dev/null || true
echo "  ✓ git-lfs 已就绪"

#-----
# 3. 配置 repo 环境变量
#-----
echo "[3/7] 配置 repo 环境变量..."
if ! grep -q 'export PATH=~/.bin:$PATH' ~/.bashrc 2>/dev/null; then
    echo 'export PATH=~/.bin:$PATH' >> ~/.bashrc
fi
export PATH=~/.bin:$PATH
echo "  ✓ PATH 已配置"

#-----
# 4. 通过 repo 下载源码
#-----
echo "[4/7] 通过 repo 下载源码（耗时较长，请耐心等待）..."
mkdir -p "$TARGET_DIR"
cd "$TARGET_DIR"

if [ ! -d .repo ]; then
    repo init -u https://gitee.com/HiSpark/manifest.git -b OpenHarmony-v5.1.0-Release -m default-ws63-without-xts.xml --no-repo-verify
fi
repo sync -c
repo forall -c 'git lfs pull'
echo "  ✓ 源码下载完成"

#-----
# 5. 下载预编译工具链
#-----
echo "[5/7] 下载预编译工具链..."
cd "$TARGET_DIR"
./build/prebuilts_download.sh
echo "  ✓ 预编译工具链下载完成"

#-----
# 6. 安装 hb 构建工具
#-----
echo "[6/7] 安装 hb 构建工具..."
cd "$TARGET_DIR"
python3 -m pip install --user build/hb -q
# pip --user 安装到 ~/.local/bin, 需加入 PATH
LOCAL_BIN="$HOME/.local/bin"
```

```

if [ -d "$LOCAL_BIN" ]; then
    export PATH="$LOCAL_BIN:$PATH"
    if ! grep -qF "$LOCAL_BIN" ~/.bashrc 2>/dev/null; then
        echo "" >> ~/.bashrc
        echo "# pip --user 安装的可执行文件" >> ~/.bashrc
        echo "export PATH=\"$LOCAL_BIN:$PATH\"" >> ~/.bashrc
        echo "  ✓ ~/.local/bin 已加入 ~/.bashrc"
    fi
fi

echo "  ✓ hb 已安装"

#-----
# 7. 安装 cmake (可选)
#-----

echo "[7/7] 安装 cmake..."
if ! command -v cmake &>/dev/null; then
    sudo apt-get install -y -qq cmake
fi
echo "  ✓ cmake 已就绪"

#-----
# 8. 配置工具链环境变量
#-----

TOOLCHAIN_PATH="$TARGET_DIR/device/soc/hisilicon/ws63v100/sdk/tools/bin/compiler/
riscv/cc_riscv32_musl_100/cc_riscv32_musl/bin"
BASHRC_ENTRY="export PATH=$TOOLCHAIN_PATH:$PATH"

if ! grep -qF "$TOOLCHAIN_PATH" ~/.bashrc 2>/dev/null; then
    echo "" >> ~/.bashrc
    echo "# ws63_ohos RISC-V 工具链" >> ~/.bashrc
    echo "$BASHRC_ENTRY" >> ~/.bashrc
    echo "  ✓ 工具链已写入 ~/.bashrc"
else
    echo "  ✓ 工具链已在 ~/.bashrc 中"
fi

#-----
# 完成提示
#-----

echo ""
echo "===== "
echo "  下载和配置完成！"
echo "===== "
echo ""
echo "请执行以下命令使环境变量生效："
echo "  source ~/.bashrc"
echo ""
echo "验证工具链："
echo "  riscv32-linux-musl-gcc -v"
echo ""
echo "【编译系统】在源码根目录执行："
echo "  cd $TARGET_DIR"
echo "  hb set -p nearlink_dk_3863"
echo "  hb build -f"
echo ""
echo "或一键完整编译（含清理）："

```

```
echo " cd $TARGET_DIR"
echo " rm -rf out && hb set -p nearlink_dk_3863 && hb build -f"
echo ""
echo "编译完成后，固件位于："
echo " $TARGET_DIR/out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-
liteos-app_all.fwpkg"
echo ""
```

三、源码下载和系统编译流程（官方流程）

3.1 代码下载

1. 安装 repo 工具

```
mkdir -p ~/bin
curl https://gitee.com/oschina/repo/raw/fork_flow/repo-py3 -o ~/bin/repo
chmod a+x ~/bin/repo
pip3 install -i https://repo.huaweicloud.com/repository/pypi/simple requests
```

2. 安装 git-lfs（用于下载大文件）

```
sudo apt-get install git-lfs
```

3. 配置 repo 环境变量

```
echo 'export PATH=~/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

4. 通过 repo 下载源码

需先创建目录，否则会下载到当前目录：

```
mkdir ws63_ohos
cd ws63_ohos
repo init -u https://gitee.com/HiSpark/manifest.git -b OpenHarmony-v5.1.0-Release
-m default-ws63-without-xts.xml --no-repo-verify
repo sync -c
repo forall -c 'git lfs pull'
```

5. 下载预编译工具链

```
./build/prebuilts_download.sh
```

6. 安装 hb 构建工具

```
python3 -m pip install --user build/hb
```

7. 安装 cmake（可选，部分组件依赖）

```
sudo apt install cmake
```

3.2 工具链配置 (WS63/nearlink_dk_3863)

编辑 `~/.bashrc`，添加（将 {源码根路径} 替换为实际路径，如 `/root/ws63_ohos`）：

```
export PATH={源码根路径}/device/soc/hisilicon/ws63v100/sdk/tools/bin/compiler/riscv/cc_riscv32_musl_100/cc_riscv32_musl/bin:$PATH
export PATH=~/.bin:$PATH
```

或使用 echo 追加（路径需替换为实际值）：

```
echo 'export PATH=/root/ws63_ohos/device/soc/hisilicon/ws63v100/sdk/tools/bin/compiler/riscv/cc_riscv32_musl_100/cc_riscv32_musl/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

执行 `riscv32-linux-musl-gcc -v` 验证工具链是否配置成功。

3.3 标准编译流程

方式一：命令行指定产品（推荐，免交互）

```
hb set -p nearlink_dk_3863
hb build -f
```

方式二：交互式选择

```
hb set
# 依次选择: mini → nearlink_dk_3863
hb build -f
```

一键完整编译（含清理）：

```
rm -rf out && hb set -p nearlink_dk_3863 && hb build -f
```

3.4 编译固件路径

```
out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/
├─ ws63-liteos-app_all.fwpkg    # 完整烧录包（用于 BurnTool）
├─ ws63-liteos-app_load_only.fwpkg # 仅应用烧录包（可选）
└─ ...
```

四、烧录流程

4.1 所需工具

工具	用途	获取方式
BurnTool	固件烧录	https://gitee.com/hihope_iot/near-link/tree/master/tools
CH340 驱动	串口驱动	https://www.wch.cn/downloads/CH343SER_EXE.html
串口终端 (XCOM 等)	查看日志	http://www.openedv.com/docs/old-products/ruanjian/ATK-XCOM.html

4.2 烧录步骤

1. **连接开发板**：USB 连接电脑，安装 CH340 驱动。
2. **打开 BurnTool**：选择对应串口，Option → 芯片选择 **WS63**（WS63E 与 WS63 同系列）。
3. **选择烧录文件**：
 - 路径示例：`out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/`
 - 文件：`ws63-liteos-app_all.fwpkg`**在 Windows 中可通过 4.3 节的路径选择烧录文件。**
4. **勾选**：`Auto Burn`、`Auto disconnect`。
5. **点击 Connect**，然后**单按开发板 RST 按键**开始烧录。
6. 烧录完成后按 **Reset** 重启开发板。
7. 打开串口工具（波特率 115200）查看启动日志。

4.3 WSL 环境下获取烧录文件

若源码在 WSL，Windows 直接通过 `\\ws1.localhost\Ubuntu-22.04\home\用户名\...` 访问，如果默认安装位置，则可通过以下打开文件

```
\\ws1.localhost\ohos_fbb_ws63\root\ws63_ohos\out\nearlink_dk_3863\nearlink_dk_3863\ws63-liteos-app\ws63-liteos-app_all.fwpkg
```

五、hb 命令详解

hb 是 OpenHarmony 的 Python 构建脚本工具，提供环境检查、产品选择、编译、清理等功能。

5.1 安装与验证

```
# 安装
python3 -m pip install --user build/hb

# 验证
hb help
```

5.2 hb set — 设置产品

设置源码根目录和要编译的产品。

用法	说明
<code>hb set</code>	无参数时进入交互式设置
<code>hb set -p <产品名></code>	直接指定产品，如 <code>hb set -p nearlink_dk_3863</code>
<code>hb set --product-name <产品名></code>	同上，长选项
<code>hb set -root <路径></code>	设置源码根目录
<code>hb set --all</code>	图形化选择更多参数（架构、debug/release、日志级别等）

示例：

```
hb set -p nearlink_dk_3863
```

关于 os_level 提示

执行 `hb set` 无参数时会进入交互式菜单，依次提示选择 **os_level**（mini/small/standard）、公司、产品等。

命令	是否提示 os_level
<code>hb set</code>	会提示
<code>hb set -p nearlink_dk_3863</code>	不会提示，os_level 自动从产品配置读取（mini）

说明：使用 `-p` 指定产品时，`os_level` 会从产品的 `config.json` 中自动获取（`type` 字段），无需手动选择。当前 `hb` 不支持 `--os-level` 等单独参数，建议始终使用 `hb set -p nearlink_dk_3863` 避免交互。

若使用 `-p` 仍出现 `os_level` 提示，可尝试重新安装 `hb`：`python3 -m pip install --user build/hb --force-reinstall`。

5.3 hb build — 编译

执行编译，支持完整编译、增量编译、单组件编译等。

常用用法

用法	说明
<code>hb build</code>	按已 set 的产品增量编译
<code>hb build -f</code>	完整编译（全量）
<code>hb build -p <产品名></code>	免 set 直接编译指定产品

用法	说明
<code>hb build <组件名></code>	单独编译某组件，如 <code>hb build kv_store</code>
<code>hb build <组件名>_test</code>	编译组件测试用例

常用参数

参数	简写	说明
<code>--full-compilation</code>	<code>-f</code>	完整编译
<code>--product-name <产品></code>	<code>-p</code>	指定产品
<code>--build-type {release,debug,profile}</code>	-	编译类型
<code>--target-cpu {arm,arm64,x86_64,riscv64,...}</code>	-	目标 CPU 架构
<code>--gn-args "key=value"</code>	-	传递 GN 参数
<code>--build-target <目标></code>	<code>-T</code>	指定单个编译目标
<code>--log-level {info,debug}</code>	-	日志级别
<code>--fast-rebuild</code>	-	跳过 prepare/preloader/gn_gen, 慎用
<code>--build-xts {True,False}</code>	-	是否编译 XTS 测试组件

示例

```
# 完整编译 nearlink_dk_3863
hb build -f -p nearlink_dk_3863

# 带 XTS 的编译
hb build -f --gn-args="build_xts=true"

# Debug 版本
hb build -f --build-type debug

# 传递 GN 参数
hb build --gn-args is_debug=true
```

5.4 hb clean — 清理

清除编译产物。

用法	说明
<code>hb clean</code>	清除当前 set 产品对应的 out 目录（保留 args.gn、build.log）

用法	说明
<code>hb clean xxx/out/xxx</code>	清除指定路径
<code>hb clean --clean-all</code>	清除所有编译产物
<code>hb clean --clean-args</code>	清除上次编译生成的 args（默认开启）
<code>hb clean --clean-out-product</code>	清除 out/{product} 目录
<code>hb clean --clean-ccache</code>	清除 .ccache 目录

5.5 hb env — 环境检查

检查或安装编译依赖。

用法	说明
<code>hb env</code>	检查当前环境是否满足编译要求
<code>hb env --check</code>	仅检查（默认）
<code>hb env --install</code>	安装缺失的依赖

5.6 hb tool — 构建辅助

查看构建目标、依赖关系等。

用法	说明
<code>hb tool --ls [out_dir] [pattern]</code>	列出匹配的构建目标
<code>hb tool --desc <out_dir> <组件></code>	显示目标详情
<code>hb tool --path <out_dir> <组件1> <组件2></code>	查找两目标间依赖路径
<code>hb tool --refs [组件 文件]</code>	
<code>hb tool --format <BUILD.gn路径></code>	格式化 .gn 文件
<code>hb tool --clean <out_dir></code>	清空输出目录（保留 args.gn）

六、编译流程内部说明

hb build 大致执行以下步骤：

1. **prepare**：准备构建环境
2. **preloader**：预加载产品配置，生成 `build_config.json`、`parts_config.json`
3. **gn_gen**：调用 GN 生成 Ninja 文件
4. **ninja**：调用 Ninja 执行实际编译
5. **post_build**：打包镜像、生成 fwpkg 等

七、常见问题

7.1 编译失败：找不到工具链

- 确认已执行 `./build/prebuilts_download.sh`
- 确认已配置 RISC-V 工具链到 PATH（见「工具链配置」）

7.2 需要完全重新编译

```
rm -rf out
hb set -p nearlink_dk_3863
hb build -f
```

7.3 烧录时无响应

- 确认串口、驱动正确
- 在点击 Connect 后**立即按 RST** 进入烧录模式
- 芯片选择 WS63（非 WS63E 等其他型号）

7.4 XTS 功能测试

选择 `nearlink_dk_3863_xts` 产品，或：

```
hb build -f --gn-args="build_xts=true"
```

7.5 hb set 提示 "Which os_level do you need?"

使用 `hb set -p nearlink_dk_3863` 可跳过该提示，`os_level` 会从产品配置自动获取。详见 5.2 节「关于 `os_level` 提示」。

八、命令速查表

场景	命令
首次完整编译	<code>hb set -p nearlink_dk_3863</code> → <code>hb build -f</code>
一键完整编译	<code>rm -rf out && hb set -p nearlink_dk_3863 && hb build -f</code>
增量编译	<code>hb build</code>
清理后重编	<code>hb clean</code> → <code>hb build -f</code>
单组件编译	<code>hb build</code> <组件名>
查看帮助	<code>hb help</code> 、 <code>hb set -h</code> 、 <code>hb build -h</code>